

PROJECT TITLE: AI-Powered Code Reviewer and Quality Assistant

PROJECT OVERVIEW:

The **AI-Powered Code Reviewer and Quality Assistant** is an intelligent static analysis and AI-enhanced code auditing system built for Python applications.

The system combines deterministic code analysis using Python's Python AST Module with advanced reasoning capabilities from the Google Gemini API to automatically evaluate code quality, detect structural inefficiencies, calculate complexity metrics, and provide AI-driven improvement suggestions.

Unlike traditional linters, this project bridges the gap between rule-based validation and human-level reasoning. It functions as a **"Senior Developer Assistant"**, enforcing best practices while offering refactoring, explanation, and test generation capabilities through an agentic workflow.

SYSTEM WORKFLOW (PIPELINE ARCHITECTURE):

The project follows a hybrid analysis pipeline:

STEP 1: CODE INGESTION

- Users upload .py files or paste raw Python code into the dashboard.
- Code can also be scanned via CLI commands.
- Session-based processing ensures temporary analysis without permanent storage.

STEP 2: STATIC ANALYSIS (AST ENGINE)

Using Python's Abstract Syntax Tree:

- Extracts functions and classes.
- Detects long methods and excessive nesting.
- Identifies basic code smells:

- Unused variables
- Missing docstrings
- Deep conditional nesting

This structural mapping provides deterministic insights before AI processing.

STEP 3: METRICS ENGINE (COMPLEXITY & MAINTAINABILITY)

Powered by Radon:

- **Cyclomatic Complexity (CC)** – Measures branching logic.
- **Maintainability Index (MI)** – Predicts overall code health.

Quality Grading System

MI Score Grade

90+	Excellent
75–89	Good
60–74	Average
< 60	Poor

These metrics help quantify technical debt.

STEP 4: AI ENRICHMENT (AGENTIC ENGINE)

Using the model:

Gemini 2.5 Flash Preview

The AI engine performs:

Refactoring

- Converts verbose logic into Pythonic structures.
- Improves readability and performance.

Code Explanation

- Provides step-by-step breakdown of complex functions.
- Helps junior developers understand logic flow.

Unit Test Generation

- Automatically generates PyTest-compatible test cases.
- Covers edge cases and expected outputs.

This stage adds reasoning beyond static rules.

STEP 5: REPORTING & ACTION

Results are categorized by severity:

- Info
- Warning
- Critical

Users can:

- Review suggestions in the dashboard.
- View side-by-side code diffs.
- Export CSV or HTML reports.
- Apply AI-based fixes directly.

MODULE BREAKDOWN (ARCHITECTURE DESIGN):

MODULE 1: Code Parsing & Analysis

- Built using ast module.
- Detects:
 - Function length
 - Nesting depth
 - Structural complexity
- Forms the structural foundation of the pipeline.

MODULE 2: AI Review Engine (Agentic Layer)

Features:

- Refactor suggestions

- Code explanation engine
- Automatic PyTest suite generation

This module introduces generative intelligence into traditional static analysis.

MODULE 3: Validation & Metrics Engine

- Cyclomatic Complexity via Radon
- Maintainability Index calculation
- Quality grading logic

Ensures objective measurement of code health.

MODULE 4: CLI & Configuration

Supported Commands:

scan → Analyze project
review → AI-enhanced review
apply → Apply suggested fixes
report → Export findings
diff → Show code changes
hook → Install Git pre-commit hook

Configuration handled through:

pyproject.toml

Allows:

- Setting max complexity
- Excluding directories
- Configuring severity thresholds

MODULE 5: VCS & CI/CD Integration

- Standalone Git pre-commit hook.
- Blocks commits if CRITICAL issues exist.
- CI templates for automated quality gates.

Enables DevOps-ready integration into development workflows.

MODULE 6: Review Web UI

Frontend built with Streamlit.

Features:

- Dark-mode dashboard
- Interactive metric displays
- Side-by-side diff comparison
- CSV export functionality
- Session-based analysis storage

The UI makes technical metrics understandable and interactive.

TECHNICAL SPECIFICATIONS:

Backend

- Python 3.x
- AST Module
- Radon
- DiffliB
- TOML

Frontend

- Streamlit
- Pandas
- Lucide Icons

AI Model

- Gemini 2.5 Flash Preview (09-2025)

Persistence

- In-memory session state
- No permanent code storage

KEY FEATURES:

- *Hybrid Static + AI Workflow
- *Maintainability Index Scoring
- *Automated Refactoring Suggestions
- *Unit Test Generation
- *CLI + Web Interface
- *Pre-Commit Hook Integration
- *CI/CD Ready

INNOVATION & IMPACT:

Traditional linters only detect rule violations.

Pure LLM tools lack deterministic structural validation.

This project combines:

- Rule-based static analysis
- Quantitative metric scoring
- LLM-powered reasoning

The result is a **hybrid agentic workflow** that significantly reduces manual review effort while improving consistency and maintainability across codebases.

CONCLUSION:

The **AI-Powered Code Reviewer and Quality Assistant** demonstrates how deterministic code analysis can be enhanced with generative AI to create an intelligent, production-ready review assistant.

By combining AST parsing, maintainability metrics, and LLM-based reasoning, the system enforces standardized coding practices while reducing human workload in the software development lifecycle.

This project showcases modern AI-assisted software engineering principles and reflects the future direction of automated code quality assurance.